

División de trabajo - Análisis semántico

Compiscript | 3 personas | Entrega: 7 de septiembre de 2026, 13:00

Dependencia obligatoria

Persona 1 entrega la base técnica. Persona 2 solo implementa sobre esa base. Persona 3 integra y completa el proyecto usando entregables aceptados de Persona 1 y Persona 2.

1. Decisiones técnicas cerradas

1. **Lenguaje y parser.** Python 3 + ANTLR4 con target Python. Se mantiene la gramática entregada y solo se modifica cuando un requisito semántico lo exige.
2. **Recorrido.** Se usa Visitor, no Listener. La razón es que el análisis necesita devolver tipos y resultados desde expresiones.
3. **Árbol.** Se construye un AST propio y reducido a partir del parse tree de ANTLR. La visualización obligatoria se genera como SVG con Graphviz.
4. **Tipos.** Tipos soportados: integer, float, string, boolean, null, arreglos T[] y clases. Se agrega float a la gramática porque el documento semántico lo exige.
5. **Operador +.** Permite suma numérica y concatenación string + string. -, *, / y % son solo numéricos. integer + float produce float.
6. **Variables.** Si hay anotación e inicializador, se valida compatibilidad. Sin anotación se infiere desde el inicializador. Una variable sin tipo y sin inicializador es error semántico.
7. **Compatibilidad.** Se permite integer -> float, subclase -> clase padre y null -> cualquier tipo declarado. El resto requiere compatibilidad exacta.
8. **Switch.** El discriminante puede ser integer, float, string o boolean; cada case debe ser compatible con el discriminante. Esta decisión sigue el ejemplo oficial de switch(x) con case 1/case 2.
9. **Arreglos.** Elementos homogéneos. El índice debe ser integer. No se implementa análisis estático de límites. Un arreglo vacío necesita contexto de tipo.
10. **Funciones.** No hay sobrecarga. Se predeclaran antes de analizar cuerpos para soportar recursión. Closures se resuelven por cadena léxica de scopes.
11. **Retornos.** Si la función declara tipo, cada return debe coincidir. Si no declara tipo, se infiere un único tipo común. No se exige análisis de todos los caminos de retorno.
12. **Clases.** Se valida padre existente, ausencia de ciclos, lookup heredado, this solo dentro de métodos/constructor, argumentos de new y firma exacta al sobrescribir.
13. **Código muerto.** Solo se reportan instrucciones posteriores a return, break o continue dentro del mismo bloque. No se construye CFG.
14. **IDE.** Flask con una sola pantalla: editor, botón Compilar, lista de diagnósticos y vista del AST. Sin autenticación, persistencia, autocompletado ni debugger.
15. **Tests.** pytest. Cada regla implementada debe tener al menos un caso válido y uno inválido. Los errores se acumulan; el compilador no se detiene en el primero.

2. Contratos entre personas

- Persona 1 expone frontend.analyze_source(source) -> FrontendResult(ast, symbols, diagnostics).
- Persona 2 expone core_semantics.analyze(frontend_result) -> CoreSemanticResult(ast, symbols, diagnostics).
- Persona 3 expone compiler_service.compile_source(source) -> CompilationResult(success, diagnostics, ast_svg).
- Nadie modifica archivos propiedad de otra persona salvo integración acordada mediante pull request.

Persona 1 - Frontend, AST, tipos y tabla de símbolos

Objetivo

Entregar una base estable sobre la que Persona 2 pueda implementar semántica sin redefinir AST, tipos, scopes ni contratos.

Tareas

- Configurar ANTLR para Python y dejar un comando reproducible para regenerar Lexer, Parser y Visitor.
- Extender Compiscript.g4 con float y FloatLiteral sin alterar otras construcciones del lenguaje.
- Crear el modelo de AST para programa, declaraciones, bloques, expresiones, funciones, clases, control de flujo, arreglos, acceso a propiedades, new y this.
- Implementar AstBuilder para convertir el parse tree de ANTLR al AST propio.
- Implementar el modelo de tipos y funciones únicas de compatibilidad: same_type, is_assignable, numeric_result y common_type.
- Implementar Symbol, VariableSymbol, FunctionSymbol, ClassSymbol y Scope.
- Implementar SymbolTable jerárquica con scopes global, función, clase y bloque; lookup hacia padres y rechazo de duplicados en el mismo scope.
- Implementar SymbolCollector que predeclare clases y funciones y luego registre variables, constantes, parámetros, atributos y métodos.
- Implementar Diagnostic(code, message, line, column) y colección de errores.
- Implementar frontend.analyze_source(source) con parseo, construcción de AST y tabla de símbolos.
- Escribir tests unitarios de gramática modificada, AST, scopes, duplicados, lookup, predeclaración, tipos y diagnóstico.

Entregables exactos

- program/Compiscript.g4
- compiler/ast_nodes.py
- compiler/ast_builder.py
- compiler/types.py
- compiler/symbols.py
- compiler/scopes.py
- compiler/symbol_collector.py
- compiler/diagnostics.py
- compiler/frontend.py
- tests/person1/**
- README sección: regenerar ANTLR y ejecutar tests de Persona 1

Gate A - Criterio de aceptación para desbloquear Persona 2

- ANTLR genera sin errores el parser y Visitor de Python.
- Un archivo .cps válido produce AST y tabla de símbolos sin diagnósticos.
- Una redeclaración, identificador duplicado o padre de scope incorrecto produce diagnóstico con línea y columna.
- Funciones y clases quedan visibles antes de analizar sus cuerpos, habilitando recursión y referencias posteriores.
- La API frontend.analyze_source está congelada y documentada.
- Todos los tests de tests/person1 pasan.

Bloqueo

Persona 2 no inicia implementación semántica hasta que Gate A esté aprobado. Puede leer el código y preparar casos de prueba, pero no crear contratos paralelos.

Persona 2 - Semántica core: expresiones, funciones y control de flujo

Dependencia

Trabaja únicamente sobre FrontendResult, AST, tipos, símbolos y scopes entregados por Persona 1.

Tareas

- Implementar CoreSemanticVisitor sobre el AST de Persona 1.
- Validar declaraciones e inicializadores de let, var y const; const debe llegar inicializada y respetar su tipo.
- Validar asignaciones a variables y constantes; prohibir reasignación de const.
- Inferir tipos de literales y expresiones primitivas y escribir el tipo inferido en cada nodo del AST.
- Validar +, -, *, /, %, &&, ||, !, ==, !=, <, <=, >, >= y operador ternario según las decisiones de tipos.
- Validar identificadores no declarados y resolver el símbolo correcto por scope.
- Validar llamadas a funciones: existencia, cantidad de argumentos, compatibilidad posicional y tipo resultante.
- Validar return dentro de función y compatibilidad con el tipo declarado o inferido.
- Soportar recursión y funciones anidadas usando los símbolos predeclarados y la cadena léxica de scopes.
- Validar condiciones booleanas de if, while, do-while y for cuando la condición exista.
- Validar break y continue solo dentro de bucles, y return solo dentro de funciones.
- Validar switch con discriminante escalar y casos compatibles con su tipo.
- Detectar código muerto posterior a return, break o continue dentro del mismo bloque.
- Detectar parámetros duplicados y funciones duplicadas según la tabla de símbolos.
- Exponer core_semantics.analyze(frontend_result) sin crear otra tabla de símbolos.
- Escribir tests válidos e inválidos para cada regla anterior.

Entregables exactos

- compiler/core_semantics.py
- compiler/core_semantic_visitor.py
- compiler/expression_rules.py
- compiler/function_rules.py
- compiler/control_flow_rules.py
- tests/person2/**
- fixtures de funciones, scopes, expresiones y control de flujo
- README sección: reglas semánticas core y ejecución de tests de Persona 2

Gate B - Criterio de aceptación para desbloquear Persona 3

- No se reconstruyen AST, scopes ni tabla de símbolos dentro de Persona 2.
- Cada nodo de expresión core queda con inferred_type cuando puede determinarse.
- Todas las reglas de variables, expresiones, funciones y control de flujo tienen un test válido y uno inválido como mínimo.
- El analizador acumula múltiples diagnósticos en una sola compilación.
- La API core_semantics.analyze está congelada y documentada.
- Todos los tests de Persona 1 y Persona 2 pasan juntos.

Bloqueo

Persona 3 no conecta IDE ni semántica de objetos hasta que Gate B esté aprobado. Esto evita integrar sobre una API inestable.

Persona 3 - Clases, arreglos, visualización, IDE e integración final

Dependencia

Consumes FrontendResult de Persona 1 y CoreSemanticResult de Persona 2. No reemplaza ninguno de esos pasos.

Tareas

- Implementar semántica de clases: clase padre existente, ciclos de herencia, búsqueda de miembros heredados y firma compatible al sobrescribir.
- Validar acceso `obj.prop` y `obj.method(...)`: existencia del miembro, tipo del atributo y firma del método.
- Validar `this` únicamente dentro de métodos y constructor de la clase actual.
- Validar `new Clase(args)`: clase existente; si hay constructor, comprobar argumentos; si no hay constructor, permitir únicamente cero argumentos.
- Validar asignación subclase -> padre y rechazar padre -> subclase sin conversión explícita.
- Implementar semántica de arreglos: literal homogéneo, arreglos anidados, arreglo vacío con contexto, índice integer y tipo del elemento resultante.
- Implementar la pasada `ExtendedSemanticVisitor` para clases, objetos y arreglos usando tipos inferidos por Persona 2.
- Implementar `ast_visualizer.py` para producir DOT y SVG de Graphviz a partir del AST propio.
- Implementar `compiler_service.compile_source(source)`, encadenando `frontend -> core semantics -> extended semantics -> visualización`.
- Implementar IDE Flask mínimo con editor, Compilar, diagnósticos y AST SVG.
- Agregar tests de clases, herencia, `this`, `new`, propiedades, métodos, arreglos y pruebas end-to-end desde código fuente hasta resultado final.
- Escribir documentación final de arquitectura y ejecución. Incluir comandos Docker/ANTLR, tests e inicio del IDE.
- Realizar integración final y resolver únicamente incompatibilidades mediante PRs pequeños y atribuibles.

Entregables exactos

- `compiler/extended_semantic_visitor.py`
- `compiler/class_rules.py`
- `compiler/array_rules.py`
- `compiler/ast_visualizer.py`
- `compiler/compiler_service.py`
- `ide/app.py`
- `ide/templates/index.html`
- `ide/static/**`
- `tests/person3/**`
- `tests/integration/**`
- `ARCHITECTURE.md`
- `README.md` final de ejecución

Gate C - Criterio de aceptación final

- El IDE compila código `.cps` sin salir de la página y muestra todos los diagnósticos con línea y columna.
- Un programa válido muestra el AST visual en SVG.
- Clases, `new`, `this`, miembros heredados y arreglos tienen tests válidos e inválidos.
- Cada requisito semántico del enunciado tiene cobertura identificable en tests.
- `pytest` completo pasa desde la raíz del repositorio.
- Un clon limpio puede regenerar ANTLR, ejecutar tests e iniciar el IDE siguiendo únicamente `README.md`.
- Los commits muestran propiedad clara por persona y no existen commits compartidos.

Integración y definición de terminado

Orden obligatorio

#	Responsable	Resultado
1	Persona 1	Frontend estable y Gate A aprobado.
2	Persona 2	Semántica core estable y Gate B aprobado.
3	Persona 3	Semántica extendida + visualización + IDE + integración.
4	Equipo	Solo corrección de defectos, documentación final y prueba de presentación.

Estructura final esperada del repositorio

```

program/Compiscript.g4
compiler/
  ast_nodes.py, ast_builder.py, types.py
  symbols.py, scopes.py, symbol_collector.py
  diagnostics.py, frontend.py
  core_semantics.py, core_semantic_visitor.py
  expression_rules.py, function_rules.py, control_flow_rules.py
  extended_semantic_visitor.py, class_rules.py, array_rules.py
  ast_visualizer.py, compiler_service.py
ide/
  app.py, templates/index.html, static/
tests/
  person1/, person2/, person3/, integration/
README.md
ARCHITECTURE.md

```

Checklist único de entrega

☐ ANTLR/parser funcional en Python.	☐ Clases, constructores, this, herencia, atributos y métodos validados.
☐ AST propio y representación visual.	☐ Arreglos e índices validados.
☐ Tabla de símbolos jerárquica para global, función, clase y bloque.	☐ Batería de tests con éxito y fallo por regla.
☐ Sistema de tipos y reglas semánticas completas.	☐ IDE para escribir y compilar Compiscript.
☐ Funciones, recursión, closures y retornos validados.	☐ Documentación de arquitectura y ejecución.
☐ Control de flujo, break/continue/return y código muerto validados.	☐ Repositorio GitHub con contribución individual clara.

Fuentes oficiales usadas

README.md de Compiscript: <https://github.com/gbrolo/compiler-2025/blob/main/compiscript/README.md>

Compiscript.g4: <https://github.com/gbrolo/compiler-2025/blob/main/compiscript/program/Compiscript.g4>

README_SEMANTIC_ANALYSIS.md: https://github.com/gbrolo/compiler-2025/blob/main/compiscript/README_SEMANTIC_ANALYSIS.md

Principio de implementación

KISS: solo lo necesario para la rúbrica. YAGNI: no agregar optimizador, intérprete, debugger, persistencia, autocompletado ni análisis de flujo avanzado. DRY: una sola representación de tipos, scopes, símbolos y diagnósticos compartida por todas las fases.